

Ministério da Saúde

Secretaria Executiva

Departamento de Informática do SUS

Coordenação-Geral de Análise e Manutenção

METODOLOGIA DE DESENVOLVIMENTO DE SOFTWARE

Documento de Arquitetura de Software

Objetivo deste Documento

Este documento tem como objetivo descrever a arquitetura do sistema FinLog, um sistema de controle financeiro desenvolvido para auxiliar usuários no gerenciamento de suas receitas e despesas. A arquitetura apresentada define a estrutura geral do sistema, seus principais componentes, as tecnologias adotadas e a forma como esses elementos se relacionam. O documento busca fornecer uma visão clara e organizada do funcionamento interno da aplicação, servindo como referência para o desenvolvimento, manutenção e evolução do sistema. Além disso, este documento tem como finalidade orientar a equipe de desenvolvimento quanto às decisões arquiteturais adotadas; garantir a padronização na implementação dos componentes do sistema; facilitar a identificação de responsabilidades entre as camadas da aplicação; apoiar futuras manutenções e possíveis expansões do sistema; servir como base para validação técnica do projeto no contexto acadêmico. O escopo deste documento abrange a definição da arquitetura do sistema, incluindo a organização em camadas, os padrões utilizados, a comunicação entre componentes e as tecnologias empregadas.

Histórico de Revisão

Data	Demanda	Autor		Descrição	Versão
13/04/2026	Diagramas	Lorrany Magalhães	Matos	Elaboração inicial dos diagramas do sistema, incluindo Diagrama de Casos de Uso, Diagrama de Classes, Diagrama de Arquitetura e Diagramas de Sequência (Login e Registro de Receita).	0.1
13/04/2026	Descrição	Teodoro Yohanna	e	Preencher o documento com as descrições apropriadas de acordo com a necessidade da seção	0.2
17/04/2026	Revisão e complementação	Renata de Oliveira Aguiar		Preencher o documento com as descrições apropriadas de acordo com a necessidade da seção e revisão da documentação	0.3

Sumário

1.	INTRODUÇÃO	3
1.1	Finalidade	3
1.2	Escopo.....	3
1.3	Definições, Acrônimos e Abreviações	3
1.4	Referências	4
2.	REPRESENTAÇÃO ARQUITETURAL	4
3.	REQUISITOS E RESTRIÇÕES ARQUITETURAIS	5
4.	VISÃO DE CASOS DE USO	5
4.1	Casos de Uso significantes para a arquitetura	5
5.	VISÃO LÓGICA	7
5.1	Visão Geral – pacotes e camadas	7
6.	VISÃO DE IMPLEMENTAÇÃO	9
6.1	Caso de Uso [00X]	9
6.1.1	Diagrama de Classes	9
6.1.2	Diagrama de Sequência.....	10
7.	VISÃO DE IMPLANTAÇÃO.....	11
8.	DIMENSIONAMENTO E PERFORMANCE	11
8.1	Volume	11
8.2	Performance	11
9.	QUALIDADE	11

1. INTRODUÇÃO

1.1 Finalidade

Este documento tem como finalidade apresentar uma visão arquitetural abrangente do sistema FinLog, utilizando diferentes perspectivas para representar seus diversos aspectos estruturais e comportamentais.

O principal objetivo é capturar, documentar e comunicar as decisões arquiteturais relevantes adotadas no desenvolvimento do sistema, fornecendo uma base sólida para compreensão, implementação e evolução da aplicação.

Para isso, o documento adota o modelo de arquitetura 4+1, proposto por Philippe Kruchten, que organiza a descrição arquitetural em múltiplas visões complementares. Esse modelo permite analisar o sistema sob diferentes pontos de vista, incluindo aspectos lógicos, de desenvolvimento, de processos e de implantação, além de integrar esses elementos por meio de cenários de uso.

A utilização desse modelo contribui para uma documentação mais estruturada, facilitando a comunicação entre os envolvidos no projeto e apoiando a validação das decisões técnicas adotadas.

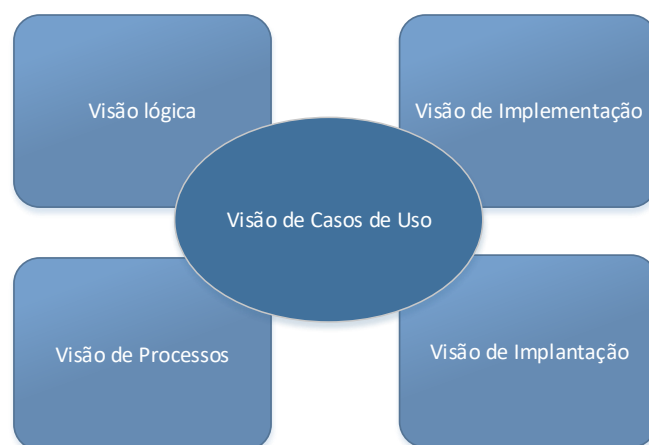


Figura 1 – Arquitetura 4+1

1.2 Escopo

O FinLog tem como foco permitir ao usuário registrar e acompanhar suas movimentações financeiras, oferecendo recursos para cadastro, edição, exclusão e visualização de receitas e despesas, além da apresentação consolidada das informações por meio de um dashboard.

Este documento também delimita o que está incluído no sistema, como o gerenciamento básico das finanças individuais, e o que não faz parte do seu escopo nesta versão, como integrações com instituições bancárias, múltiplos usuários com diferentes níveis de acesso ou funcionalidades avançadas de investimento.

Dessa forma, busca-se estabelecer uma compreensão clara sobre os limites funcionais do sistema, servindo como referência para o desenvolvimento e evitando interpretações ambíguas sobre suas capacidades.

1.3 Definições, Acrônimos e Abreviações

1.1 Referências

[KRU41]: The “4+1” view model of software architecture, Philippe Kruchten, November 1995, <http://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf>

[QOS] <https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html>

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

Visão	Público	Área	Modelo da MDS
Lógica	Analistas	Realização dos Casos de Uso	Diagrama de Classes; Diagrama de Sequência
Processo	Integradores	Performance, Escalabilidade, Concorrência	Diagrama de Atividades; Diagrama de Sequência
Implementação	Programadores	Componentes de Software	Diagrama de Componentes
Implantação	Gerência de Configuração	Nodos físicos	Diagrama de Implantação
Caso de Uso	Todos	Requisitos funcionais	Diagrama de Casos de Uso
Dados	Especialistas em dados Administradores	Persistência de dados	Modelo Entidade-Relacionamento (MER); Diagrama de Classes

	de dados		
--	----------	--	--

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

A tabela abaixo descreve a relação entre os principais requisitos do sistema FinLog e as soluções tecnológicas adotadas para atendê-los. São abordados aspectos fundamentais do desenvolvimento da aplicação, incluindo a definição das tecnologias utilizadas, a plataforma de execução, os mecanismos de segurança, a estratégia de persistência de dados e o suporte à internacionalização.

Requisito	Solução
Linguagem	Desenvolvimento do backend utilizando Java com o framework Spring, responsável pela construção da API e regras de negócio. O frontend será desenvolvido com a biblioteca React, permitindo a criação de interfaces dinâmicas e responsivas.
Plataforma	A aplicação será executada em ambiente de servidor web, possibilitando o acesso via navegador. Para armazenamento de dados, será utilizado o sistema gerenciador de banco de dados relacional MySQL.
Segurança	Serão implementados mecanismos de autenticação de usuários baseados em credenciais (e-mail e senha), com armazenamento seguro por meio de técnicas de hash criptográfico, garantindo a proteção de dados sensíveis.
Persistência	A aplicação contemplará operações completas de persistência (CRUD), permitindo o gerenciamento das informações financeiras. Também serão adotadas estratégias de modelagem de dados em banco relacional, além de mecanismos de backup e recuperação para garantir a integridade e disponibilidade das informações.

Internacionalização (i18n)	A aplicação oferecerá suporte à internacionalização, permitindo a adaptação a diferentes regiões. Inicialmente, será priorizada a formatação correta de valores monetários conforme a localidade do usuário, assegurando a exibição adequada das informações financeiras.
-----------------------------------	---

Tabela 2 – Requisitos e restrições

4. VISÃO DE CASOS DE USO

A seguir, todos os casos de uso identificados para o FinLog, com base no Diagrama de Casos de Uso (Figura 2) e nas Especificações de Casos de Uso:

ID	Nome	Descrição Resumida	Ator
UC01	Efetuar Login	Permite ao usuário autenticar-se no sistema inserindo e-mail e senha, gerando um token de sessão seguro.	Usuário do Aplicativo
UC02	Efetuar Logout	Encerra a sessão do usuário de forma segura, invalidando o token e removendo dados de sessão do cliente.	Usuário do Aplicativo
UC03	Registrar Receita	Permite o cadastro de uma entrada financeira com valor, data, categoria e descrição, atualizando o saldo automaticamente.	Usuário do Aplicativo
UC04	Registrar Despesa	Permite o registro de uma saída financeira categorizada, atualizando o saldo em tempo real.	Usuário do Aplicativo
UC05	Editar Receita	Permite a modificação dos dados de uma receita já cadastrada, com recálculo automático do	Usuário do Aplicativo

		saldo.	
C06	Editar Despesa	Permite a alteração de informações de uma despesa existente, com recálculo automático do saldo.	Usuário do Aplicativo
UC07	Excluir Receita	Remove permanentemente uma receita cadastrada após confirmação do usuário, atualizando o saldo.	Usuário do Aplicativo
UC08	Excluir Despesa	Remove permanentemente uma despesa cadastrada após confirmação do usuário, atualizando o saldo.	Usuário do Aplicativo
UC09	Visualizar Dashboard	Exibe um resumo consolidado das finanças do usuário: saldo total, receitas, despesas e possíveis gráficos/indicadores.	Usuário do Aplicativo
UC10	Filtrar Movimentações	Permite filtrar receitas e despesas por período, categoria ou tipo para análise do comportamento financeiro.	Usuário do Aplicativo
UC11	Criar Meta Financeira	Permite ao usuário definir objetivos de economia com valor alvo, prazo e descrição, salvando-os no banco de dados.	Usuário do Aplicativo

4.1 Casos de Uso significantes para a arquitetura

Nem todos os casos de uso impactam igualmente as decisões arquiteturais. Os casos de uso a seguir foram selecionados por exercitarem os principais mecanismos arquiteturais do sistema (autenticação, persistência, validação, cálculo de saldo e apresentação de dados) e por possuírem prioridade Crítica ou Importante conforme o Documento de Visão (seção 5 – Precedência e Prioridades):

ID	Nome	Justificativa Arquitetural	Prioridade
UC01	Efetuar Login	Ponto de entrada de todo o sistema. Define o mecanismo de autenticação e gestão de sessão que impacta todos os demais casos de uso.	Crítico
UC03	Registrar Receita	Funcionalidade central do MVP. Demonstra o fluxo completo: Interface → Backend → Banco de Dados, com validação e atualização de saldo.	Crítico
UC04	Registrar Despesa	Análogo ao registro de receita, mas representa a principal necessidade do usuário: controle de gastos. Impacta diretamente o saldo exibido.	Crítico
UC09	Visualizar Dashboard	Consolida os dados de receitas, despesas e saldo, exercitando a lógica de cálculo e a camada de apresentação de dados ao usuário.	Crítico
UC11	Criar Meta Financeira	Representa um módulo de dados independente (MetaFinanceira) que agrega valor ao produto e valida a extensibilidade da arquitetura.	Importante



Figura 2 – Exemplo de Diagrama com os casos de uso significativos e atores

5. VISÃO LÓGICA

Esta seção descreve a visão lógica da arquitetura do FinLog, apresentando a organização do sistema em camadas e pacotes, os casos de uso arquiteturalmente significantes e os principais aspectos dinâmicos do sistema.

5.1 Visão Geral – pacotes e camadas

O FinLog adota uma arquitetura em três camadas, conforme representado no diagrama de camadas (Figura 2) presente neste documento. Cada camada possui responsabilidades bem definidas e utiliza tecnologias específicas, conforme descrito a seguir:

Camada	Responsabilidade	Tecnologia
Frontend (Interface)	Responsável pela renderização da interface gráfica, captura das entradas do usuário, validação inicial de formulários e comunicação com o backend por meio de requisições HTTP.	Navegador web moderno (HTML, CSS e JavaScript) com uso da biblioteca React
Backend (Aplicação)	Responsável pelo processamento da lógica de negócio, autenticação e autorização de usuários, validação de dados, cálculo de saldo e disponibilização de serviços ao frontend.	Java (runtime) com uso do framework Spring
Banco de Dados	Responsável pela persistência das informações do sistema, incluindo usuários, receitas, despesas, categorias e histórico de transações.	MySQL, com scripts SQL versionados e uso de pool de conexões

O fluxo padrão de uma operação no sistema segue a sequência: Usuário → Interface (Frontend) → Backend → Banco de Dados → Backend → Interface → Usuário

Nesse fluxo, o usuário interage com a interface, que envia requisições ao backend. O backend processa a solicitação, interage com o banco de dados quando necessário e retorna a resposta ao frontend, que atualiza a interface apresentada ao usuário.

Esse padrão de interação está representado no diagrama de pacotes (Figura 3), também presente neste documento.

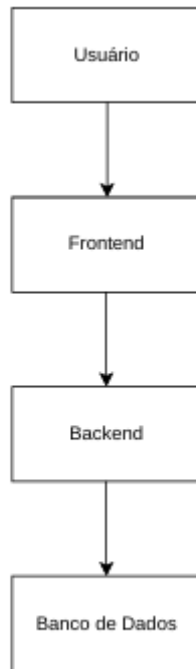


Figura 2 – Diagrama de Camadas da Aplicação

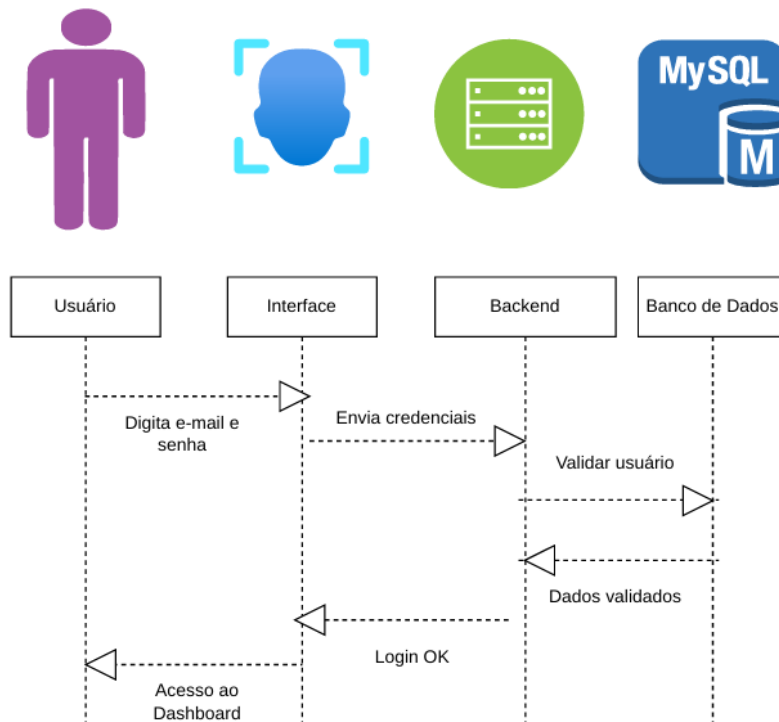


Figura 3 – Diagrama de Pacotes da Aplicação

6. VISÃO DE IMPLEMENTAÇÃO

Esta seção descreve a organização dos principais componentes de software do **FinLog**, detalhando as classes, seus relacionamentos e os fluxos de interação entre os elementos do sistema, com foco nos casos de uso mais relevantes do ponto de vista arquitetural.

Esses casos de uso orientam a estrutura da aplicação, influenciando a definição das responsabilidades das classes e a comunicação entre as camadas do sistema.

6.1 Caso de Uso Identificados

O sistema FinLog possui um único ator principal: o Usuário do Aplicativo. A partir da análise dos documentos de visão e especificações de casos de uso, os seguintes casos de uso foram identificados:

- UC01 - Efetuar Login: Permite ao usuário acessar o sistema por meio de suas credenciais, garantindo autenticação segura.
- UC02 - Efetuar Logout: Permite ao usuário encerrar sua sessão no sistema de forma segura, protegendo seus dados.
- UC03 - Registrar Receita: Permite ao usuário cadastrar uma entrada financeira, informando valor, data, categoria e descrição.
- UC04 - Registrar Despesa: Permite ao usuário registrar uma saída financeira, possibilitando o controle de gastos.
- UC05 - Editar Receita: Permite ao usuário alterar informações de uma receita previamente cadastrada.
- UC06 - Editar Despesa: Permite ao usuário atualizar os dados de uma despesa existente.
- UC07 - Excluir Receita: Permite ao usuário remover uma receita do sistema.
- UC08 - Excluir Despesa: Permite ao usuário excluir uma despesa registrada.
- UC09 - Visualizar Dashboard: Permite ao usuário visualizar um resumo geral das suas finanças, incluindo saldo, receitas e despesas.
- UC10 - Filtrar Movimentações: Permite ao usuário aplicar filtros por período, categoria ou tipo, facilitando a análise financeira.
- UC11 - Criar Meta Financeira: Permite ao usuário definir metas financeiras, auxiliando no planejamento e controle de objetivos.

6.1.1 Diagrama de Classes

O modelo de classes do FinLog é composto pelas seguintes entidades principais, conforme identificado no documento de arquitetura:

Classe	Atributos	Relacionamentos
Usuário	id, nome, email, senha	1:N com Receita; 1:N com Despesa; 1:N com MetaFinanceira
Receita	id, valor, data, descrição	N:1 com Usuário; N:1 com Categoria
Despesa	id, valor, data, descrição	N:1 com Usuário; N:1 com Categoria
Categoria	id, nome	1:N com Receita; 1:N com Despesa
MetaFinanceira	id, valorMeta, prazo, descrição	N:1 com Usuário

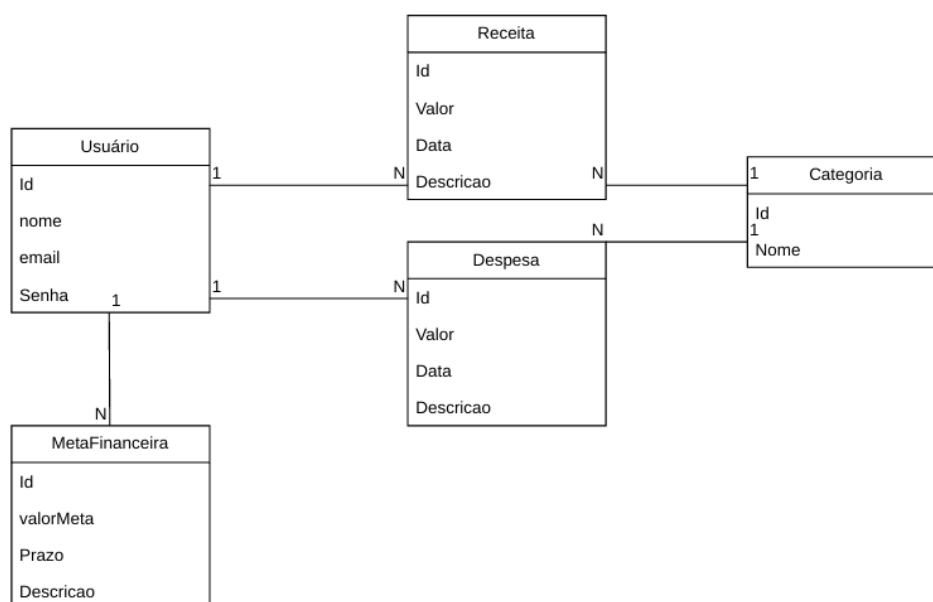


Figura 20 – Diagrama de Classes

6.1.2 Diagrama de Sequência

Os diagramas de sequência descrevem o fluxo de mensagens e interações entre os componentes do sistema FinLog, evidenciando a ordem em que as operações são executadas para atender aos casos de uso mais significativos. Esses diagramas permitem compreender o comportamento dinâmico da aplicação, destacando como as requisições são iniciadas pelo usuário, processadas pelas camadas do sistema e respondidas de forma estruturada. Os principais participantes envolvidos nesses fluxos são:

- Usuário, responsável por iniciar as interações com o sistema;
- Interface (Frontend), desenvolvida com React, responsável por capturar as ações do usuário e enviar requisições ao backend;
- Backend, implementado com Spring Boot, responsável pelo processamento da lógica de negócio e comunicação com o banco de dados;
- Banco de Dados, utilizando MySQL, responsável pela persistência das informações.

Dessa forma, os diagramas de sequência ilustram o fluxo completo das operações, desde a interação inicial do usuário até a resposta final exibida na interface.

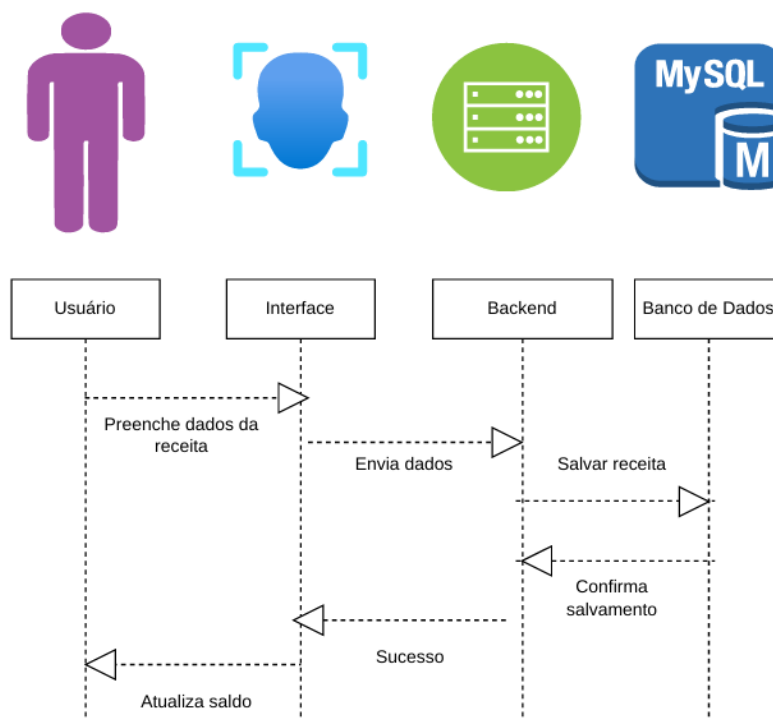


Figura 20 – Diagrama de Sequência

7. VISÃO DE IMPLANTAÇÃO

Esta seção apresenta a visão de implantação do sistema FinLog, descrevendo os nós físicos e ambientes envolvidos na execução da aplicação, bem como suas configurações e os artefatos de software distribuídos em cada componente.

São detalhados os elementos da infraestrutura, incluindo servidor de aplicação, cliente (navegador) e banco de dados, além da forma como esses componentes se comunicam em ambiente de execução. Também são considerados aspectos como distribuição da aplicação, configuração dos serviços e dependências necessárias para o correto funcionamento do sistema.

Essa visão tem como objetivo evidenciar como o sistema é implantado e executado no ambiente real, contribuindo para a compreensão da infraestrutura necessária e apoiando atividades de configuração, manutenção e escalabilidade da aplicação.

7.1 Nós Físicos

Nodo	Descrição	Tecnologia
Máquina do Desenvolvedor	Ambiente local utilizado para desenvolvimento e testes da aplicação. Possui as ferramentas necessárias para codificação, versionamento e execução do sistema.	IDE compatível com Java, Git, Maven ou Gradle
Servidor de Aplicação	Responsável pelo processamento da lógica de negócio, autenticação de usuários e disponibilização das APIs para o frontend.	Java (runtime) com Spring Boot e servidor web embutido
Servidor de Banco de Dados	Responsável pelo armazenamento e gerenciamento das informações do sistema, incluindo receitas, despesas, metas e histórico de transações.	MySQL, scripts SQL versionados e pool de conexões

Cliente (Navegador)	Ambiente onde o usuário acessa o sistema por meio de uma URL. O frontend é carregado e executado no navegador.	Navegador web moderno com suporte a HTML, CSS e JavaScript, acesso à internet
---------------------	--	---

7.2 Repositório e Distribuição

O código-fonte do sistema FinLog será versionado e disponibilizado em repositório no GitHub, utilizando boas práticas de controle de versão, como a organização por branches e o uso de tags para identificação de versões estáveis.

As instruções necessárias para build, execução e configuração do ambiente de desenvolvimento serão documentadas no arquivo README do repositório, garantindo que novos desenvolvedores consigam configurar e executar o projeto de forma adequada.

Além disso, os scripts relacionados ao banco de dados, incluindo migrações e possíveis procedimentos de rollback, serão mantidos no próprio repositório e versionados em conjunto com o código-fonte, assegurando consistência entre a aplicação e sua estrutura de dados.

7.3 Pré-requisitos de Implantação

Para a correta implantação e execução do sistema FinLog, é necessário que os seguintes requisitos sejam atendidos:

- Acesso à internet, permitindo que o usuário final utilize o sistema por meio de um navegador web (acesso via URL).
- Disponibilidade de uma instância ativa do sistema gerenciador de banco de dados MySQL para armazenamento e gerenciamento das informações.
- Ambiente de execução com suporte à linguagem Java, incluindo o runtime necessário para a execução do backend da aplicação.
- Ambiente de desenvolvimento devidamente configurado, contendo uma IDE compatível com Java, além das ferramentas e frameworks definidos pela equipe para construção e manutenção do sistema.

8. DIMENSIONAMENTO E PERFORMANCE

Esta seção apresenta estimativas de volume de uso e os requisitos de performance esperados para o sistema FinLog, com base nas definições estabelecidas no Documento de Visão e nos Requisitos Não-Funcionais.

8.1 Volume

Item	Estimativa
Número estimado de usuários (MVP)	[novo] Não definido nos documentos de referência. Recomenda-se definir com base no público-alvo do MVP.
Número estimado de acessos diários	[novo] A ser definido após lançamento do MVP e coleta de métricas reais de uso.
Número estimado de acessos por período	[novo] A ser definido com base em dados de uso após lançamento.
Tempo de sessão estimado por usuário	[novo] A ser definido com base em análise de comportamento pós-lançamento.
Plataforma de uso prevista	Computadores (desktops e laptops) acessando via navegador web.

8.2 Performance

Requisito de Performance	Meta / Valor
--------------------------	--------------

Tempo máximo de resposta por transação (RNF005)	200ms para operações comuns (registro, edição, exclusão de receitas e despesas).
Disponibilidade e backup automático (RNF004)	O sistema deve realizar cópias de segurança automaticamente, garantindo recuperação dos dados em caso de falha.
Proteção contra erros do usuário (RNF001)	O sistema deve impedir a entrada de dados inconsistentes ou inválidos, com validação no Frontend e no Backend.
Mensagens de erro padronizadas (RNF006)	Erros devem ser informados ao usuário de forma clara, compreensível e uniforme.
Tempo de carga do Dashboard	[novo] Não especificado nos documentos de referência. Recomenda-se definir meta (ex.: inferior a 1 segundo) para o MVP.

9. QUALIDADE

Esta seção apresenta os principais atributos de qualidade considerados no desenvolvimento do sistema FinLog, com base nos requisitos não funcionais definidos no Documento de Visão e nos critérios estabelecidos pela norma ISO/IEC 25010:2011.

Os atributos de qualidade descrevem características essenciais que o sistema deve atender, como desempenho, segurança, usabilidade, confiabilidade e manutenibilidade. Esses aspectos orientam as decisões arquiteturais adotadas, garantindo que a aplicação não apenas funcione corretamente, mas também ofereça uma experiência adequada ao usuário e seja sustentável ao longo do tempo.

Item	Descrição	Solução
------	-----------	---------

Segurança (RNF003)	Cada usuário deve possuir um perfil protegido por senha. Os dados financeiros são sensíveis e devem ser protegidos contra acesso não autorizado, em conformidade com a LGPD.	Hash seguro para senhas; criptografia de campos sensíveis e de dumps de backup; sessão mantida via token/cookie seguro.
Usabilidade (RNF001, RNF005, RNF006)	O sistema deve ser simples, intuitivo e acessível para diferentes perfis de usuário. Deve proteger contra erros de entrada, responder rapidamente e apresentar mensagens de erro claras.	Validação de campos no Frontend e no Backend; tempo de resposta $\leq 200\text{ms}$; mensagens de erro padronizadas; interface responsiva e acessível.
Manutenibilidade (RNF002)	O sistema deve ser desenvolvido de forma modular e possuir documentação clara, facilitando futuras evoluções e manutenções.	Código modular com separação de camadas (Frontend, Backend, Banco de Dados); documentação no repositório (README, script versionados, logs estruturados).
Disponibilidade (RNF004)	O sistema deve garantir a continuidade do serviço e a recuperação dos dados em caso de falha.	Backups automáticos com scripts de dump/restore versionados no repositório; rotação e retenção configurável de logs.
Confiabilidade	O sistema deve garantir a integridade das informações financeiras dos usuários, evitando perda ou corrupção de dados.	Suite de testes automatizados (unitários e de integração) rodando contra instância MySQL; versionamento de esquema com scripts de migração e rollback.

Portabilidade	[novo] Não explicitado como requisito nos documentos de referência, mas relevante dado o ambiente web.	[novo] O sistema roda em qualquer navegador moderno com acesso à internet, sem necessidade de instalação local pelo usuário final.
Escalabilidade	[novo] Não definida no escopo do MVP, mas relevante para versões futuras do produto.	[novo] A arquitetura em camadas (Frontend / Backend / Banco de Dados) facilita o escalonamento horizontal do backend e a migração para serviços gerenciados de banco de dados em versões futuras.
Conformidade Legal	O sistema deve atender à LGPD (Lei nº 13.709/2018) e ao Marco Civil da Internet (Lei nº 12.965/2014), garantindo proteção, privacidade e portabilidade dos dados dos usuários.	Permissão para exclusão de dados pessoais; proteção de dados sensíveis; registro e guarda de logs de acesso; garantia de integridade e confidencialidade das comunicações.

A qualidade do MVP será avaliada com base nos atributos mínimos definidos pela ISO/IEC 25010:2011, servindo como referência para as métricas de aceitação do sistema.